

Part A: Complexity Analysis

Problem 1: Determine the time complexity for each code snippets with justification:

```
int hello_recursive(int n) {  
    if (n <= 1) return 1;  
    return hello_recursive(n-1) + hello_recursive(n-2);  
}
```

Time Complexity: **$O(2^n)$**

More specifically $O(\Phi^n)$ where Φ is the golden ratio.

Level 0: $f(n)$ (1 node)

Level 1: $f(n-1), f(n-2)$ (2 nodes)

Level 2: $f(n-2), f(n-3), f(n-3), f(n-4)$ (4 nodes)

At level k number of nodes would be at most 2^k

The height of this tree would be maximum of n

$n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots \rightarrow 1$ (the very left path)

Total number of Nodes (Function calls) would be

$$2^0 + 2^1 + 2^2 + \dots + 2^n = 2^{n+1} - 1$$

1 function call performs $O(1)$

So over all time complexity would be **$O(2^n)$**

NOTE: We lost the tight upper bound here by assuming perfect binary tree which is not true (but we are estimating the upper bound so it's ok).

```

void test(int n) {
    int i = n;
    while (i > 1) {
        int j = 1;
        while (j < i) {
            j *= 2;
        }
        i /= 2;
    }
}

```

Inner loop runs $\log(i)$ times

Where i divides each time until it reaches 1

$n, n/2, n/4, \dots, 1$

So the time complexity would be $\log(n) + \log(n/2) + \log(n/4) + \log(1) + \dots$ or $\log(n/2^k)$

it stops when $i = 1$ ($n/2^k = 1$, where $k = 0, 1, 2, 3, \dots$)

so $k = \log_2(n)$

$$\sum_{k=0}^{\log_2 n} \log_2\left(\frac{n}{2^k}\right) = \sum_{k=0}^{\log_2 n} (\log_2 n - k)$$

$$\log_2(n) + \log_2(n) - 1 + \log_2(n) - 2 + \dots + 1 + 0$$

This is actually the sum of the first x integers where $x = \log_2(n)$

It's like

$$x + x-1 + x-2 + \dots + 1$$

Which is equal to $k(k+1)/2$

$$\log_2(n) * (\log_2(n) + 1)/2 = ((\log_2(n))^2 + \log_2(n))/2$$

So time complexity would be **$O(\log^2(n))$**